Clases del proyecto Ooodle (práctica aplicada a sistemas)

Vista:

- **RegistroVista:** Esta clase se encarga de presentar la interfaz de registro al usuario, permitiéndole ver dos campos donde podrá ingresar su nombre de usuario y contraseña. Un botón para acabar su registro y otro botón para ir directamente al inicio de sesión si ya posee una cuenta. Si el usuario deja espacios vacíos o ingresa un nombre que ya se encuentra en el sistema, le presentará una ventana emergente con un mensaje que le menciona claramente cual fue el error en su registro (nombre ya existente, nombre vacío, contraseña vacía), permitiéndole volver a realizar el registro luego de avisarle. Si el registro es éxitoso, le mostrará igualmente una ventana emergente que le confirmará su registro y automáticamente enviará al usuario a la pestaña de inicio de sesión.

- **InicioSesionVista:** Esta clase se encarga de presentar la interfaz de inicio de sesión al usuario, con dos espacios vacíos donde colocará su nombre y contraseña anteriormente registrados. El usuario tendrá dos botones, uno que le permite iniciar sesión una vez haya rellenado los espacio y otro que le permite ir a la pestaña de registro sino lo ha hecho anteriormente. Si el usuario tiene un error en el momento de iniciar sesión, le mostrará una ventana emergente indicándole “usuario o contraseña incorrecta”, permitiéndole volver a intentar iniciar sesión.

- **Principal:** Esta clase funciona como la clase main, se encarga de la presentación de las interfaces de registro e inicio de sesión.

Controller:

- **RegistroController:** Esta clase contiene toda la lógica detrás del registro. Revisa que los espacios de nombre y contraseña no se encuentren vacíos, si se da el caso llama a la clase CasosRegistro y le agrega un mensaje único para cada caso (NombreVacio, ContrasenaVacia), también si el usuario ya se encuentra registrado (UsuarioYaExiste). Esto último gracias a los datos

guardados en la clase UsuarioDAO que es llamada para comprobar datos y verificar si ese nombre ya se encuentra en la lista de registros. Si ninguno de estos casos se presenta, el registro del usuario es aceptado y le envía un mensaje de registro exitoso (RegistroExitoso).

- **InicioSesionController:** Maneja la lógica detrás del inicio de sesión. Llamando a la clase UsuarioDAO, verifica si el usuario ya se encuentra registrado. Si ese es el caso, permite el inicio de sesión, de lo contrario no lo permite. Dependiendo del resultado, le avisará a la clase InicioSesionVista si debe mostrar el mensaje de bienvenida o avisarle al usuario que algo se encuentra mal.

Servicios

- **UsuarioDAO:** Esta clase guarda toda la información relacionada al registro (nombre, contraseña) en una lista, impidiendo que esto suceda si algún nombre se llega a repetir. Si sucede esto, el segundo intento de registro con ese nombre no se permitirá y le dirá a la clase RegistroController que no se permite su registro y que esta le diga a RegistroVista que le muestre al usuario "Usuario ya existente". De lo contrario, el registro ocurrirá sin problemas y los datos se guardarán en la lista. Los datos guardados en esta lista serán utilizados por RegistroController para comprobar si el nombre ha sido usado con anterioridad o no, también le permite a la clase InicioSesionController verificar los datos y si los datos ingresados se encuentran en la lista, le permite iniciar sesión al usuario. Cabe recalcar que cada contraseña queda adjudica al usuario que se registró. Por ejemplo, el usuario Lony crea una cuenta con la contraseña 1234, mientras en el inicio de sesión ambas credenciales se escriban juntas, permitirá el inicio de sesión. De lo contrario, si una nueva cuenta con el nombre de usuario David, con la contraseña 2104 intenta ingresar y por algún casual intenta iniciar sesión con la contraseña de Lony (1234) no podrá hacerlo, pues esta pertenece al usuario Lony y no al usuario David. También es necesario recalcar que varios usuarios pueden tener la misma contraseña y esto no se verá afectado en la lista. Si se llega a dar el caso de que un usuario use la contraseña Politecnico123* y otro también, el inicio de sesión dependerá del nombre del usuario. En pocas palabras, la lista guarda nombres que deben ser diferentes, pero no realiza esta verificación con contraseñas.

- **LogerSimple:** Esta clase se encarga de guardar los errores ocurridos durante la ejecución del programa (espacios vacíos, datos incorrectos, registros con nombre ya existentes) y los guarda en un documento que permite al desarrollador verificarlo después.

Modelo

- **Usuario:** Esta clase contiene los atributos que pertenecen a un usuario (nombre, contraseña) y permite crear objetos de tipo usuario que son necesarias en otras clases, como es el caso de la clase UsuarioDAO.
- **CasosRegistro:** Esta clase genera una constante de casos que se puedan presentar durante el registro del usuario, para nuestro caso se utilizan las constantes NombreVacio, ContrasenaVacia, UsuarioYaExiste, RegistroExitoso. Estos muestran directamente un aviso al usuario, por ello se usa en la clase RegistroVista pues se encargan de mostrar el mensaje necesario para cada caso. Todo esto se hace en base a enum's, estos tipos de datos permiten unos resultados fijos en unas constantes (que son las anteriormente mencionadas).